

**If you want to
automate, you learn
to code**

- and you learn to code well

Fast paced summary of what we are going to cover

insert fast paced summary here :)

Alan Richardson

- www.javafortesters.com
- www.seleniumsimplified.com
- www.eviltester.com
- www.compendiumdev.co.uk
- uk.linkedin.com/in/eviltester
- [@eviltester](#)

"Automate" doesn't mean "Automate Testing"

- [dictionary.com/browse/automate](https://www.dictionary.com/browse/automate)

"to install automatic procedures, as for manufacturing or servicing;"

**You don't *need* to
'code' to 'automate'**

We can Automate by adding tools

I use Tools

- FTP**

- Text Editors**

- etc.**

**We take 'tools' for
granted**

Because we use them so often

**There are some
processes we have to
automate**

**And someone else might have
automated them already.**

That's a tool.

**But tools come with
limitations**

Requisite Variety

"Only variety can destroy variety" - Ross Ashby

[en.wikipedia.org/wiki/Variety \(cybernetics\)\)](https://en.wikipedia.org/wiki/Variety_(cybernetics))

"Only variety can absorb variety" - Stafford Beer

Strategy #1 - Build a toolbox with Multiple Tools

**There is a difference
between 'Strategic'
and 'Tactical'**

Automate Tactically

solve a problem quickly

Automate Strategically

solve a problem for the long term

**Toolbox building is
'Strategic'**

**A long term strategy to identify
tools.**

And I have to commit to this.

**Strategies change and
evolve**

Strategic Vs Tactical

Finding Tools can be a Tactical pursuit

**If I learn how to find tools.
I can find tools when I need them.**

**Finding Tools when
you need them**

**Is also a key step in learning how
to automate.**

How do we find tools?

- What are you trying to achieve?
 - learn the domain language for the task
- Narrow by technology?
 - technology used
 - technology tool implemented in

Non-domain specific task

**Search: "I want to copy a
document to another machine"**
learn: server, transfer, file

Domain language for task

Search: "Tool to transfer file to server"

learn: ftp, scp, sftp, ssh protocol, login, user account

Specific Technology

"What does my specific server use?"

"What do I use?"

**Search: "Windows Software to
transfer file via scp ssh key"**

Building these queries and chaining them is a skill

- the more we learn the better we get
- the better we understand the technology, the faster we find what we need
- technical staff take this skill for granted
- we also use this skill to find - IDE, Libraries, Build Tools, etc.

Extend tools

Assuming that you can find tools

Favour tools that support extension

Learn the Extension possibilities for your chosen tools

**Scripting a tool is an
easy way to learn to
code**

Tool Chaining

**Don't underestimate the power
of the command line
On Windows and Linux**

**And now for the meat
of of it**

**"Give a man a fish you feed him
for a day.**

**Teach a man to fish then you feed
him for a lifetime."**

The Ultimate Aim

**"Give humankind the power to
summon forth fish from the
empty air"**

**Learn to code and
write our own**

**When people learn to
code they focus on**

**programming language
IDEs**

**build process
'making programs'**

**We don't always
'make programs'
when we automate**

**Pick your
programming
language for the right
reasons**

'Easy to learn'?

'Easy to run code'?

'Easy to cobble together stuff that runs from copy and pasted examples off the internet'?

How to make Java Easy to run

**Write an @Test method that does
stuff**

**Run the @Test method
Tadah!**

So how do we pick a language?

- what is your system written in?
- what languages do people that you can ask for help know how to code?
- what examples have you found that do what you want to do?
- what tutorials or books have you found that you like?

Automating Tactically Vs Automating Strategically

"Test Automation" - Automating Strategically

- long term
- paths through system

Automate Tactically

- Automate to save time
- Automate to make your process more repeatable
- Automate to help you, or someone else

Tactical Solutions Might Become Strategic

- Maintainable
- Abstraction Layers
- Tested

Strategic Solutions can be used Tactically

- Adhoc exploration using abstraction layers
- Temporary Data Exploration

Automate Tactically With

- Tools
- Chaining of Tools
- Scripting languages using tools and libraries
- @Test method using libraries

Warning - Tactical Solutions might not translate to Strategic Solutions without work

- the tactics we choose might not scale
- unreliable interaction
- time based synchronisation
- hacky code

Danger - Opportunity Cost

- Some people view "Testing" as an opportunity cost of increased strategic "Automating"
- We could also consider "Not improving your ability to automate" an Opportunity cost of not automating strategically

**We can refactor
tactical solutions into
strategic solutions
if we chose appropriate
technology**

Examples of moving from tactical to strategic

Can't 'show you' many because:

- a lot of this is commercially sensitive
- very domain and project specific

How do I tend to build tactical solutions?

I can tell you that:

- I tend to use Java, not a scripting language
- Because... libraries, maven, JUnit
 - ... I know it well

And now an Example

Pandocifier

Automate Tactically

- batch shell scripts to automate pandoc
- pandocifier to run leanpub through pandoc

Refactor to Strategically Automated

- review, plan, refactor
- keep it working as a tactical automated solution

Review Pandocifier

- Refactor to domain objects
- Execute Pandoc
- Inject Configuration
- Externalise Configuration
- External Pandocifier Object
- Build into app

Refactor to domain objects

- There is one domain object - `BookTxtFile`
- If I refactor to domain objects
 - fewer comments in the code
 - fewer lines of code in tactical solution
- e.g. `PreviewMdFile`, `BookDetailsContentReader`
- write Unit tests for the domain objects

Execute Pandoc

Instead of writing out instructions to the user to execute 'tactic'.

```
// output the command to generate the book to console
System.out.println(
    "pandoc leanpubpreview.md -f markdown -s " +
    "-o leanpubpreview.pdf --toc");
```

The code could execute them:

```
Process p = Runtime.getRuntime().exec
    ("pandoc leanpubpreview.md -f markdown -s " +
    "-o leanpubpreview.pdf --toc")
```

Inject Configuration

Instead of hardcoding paths and file names (possibly pandoc arguments)

I could create a `PandocifierConfig` object with `getInputFilePath()` and `getOutputFileName()`

Externalise Configuration

Mechanism to create the `PandocifierConfig` from a file

- Properties file - easy to read with standard Java
- text file - write a parser
- JSON file - need an external library

External Pandocifier Object

- could create Pandocifier from @Test injecting config
- `new Pandocifier(config).createPDF()`

Build into app

- create a main method with params that setup config
- use `mvn package` to create a `.jar` file
- use `java -jar pandocifier.jar` to execute
- add to path and run from anywhere

Prioritise the 'ideas' to maximise tactical advantage

- Refactor, Execute, ...
- Inject Config, Execute Pandoc, ...
- Externalise Configuration, Execute Pandoc ...

Decide Prioritisation

- Inject Config,
- Execute Pandoc,
- Pandocifier,
- External Config

find the [code to support this decision on github](#)

Walkthrough of Changes for Config

- Create object to model config - store, not transform
- I Know that 'if' I create a main later, I can still use this
- Move the 'hard coded' stuff into an object
- Add it to an @Before to 'simulate' injection
- Possible future need for a PanDoc configuration

see [LeanPubConfigTest.java](#)

Walkthrough of Changes for Execute

- Initially `Runtime.getRuntime().exec("...")`.
- Execution errors
- Rather than debug it I moved to `ProcessBuilder`
- New config for the `setPandocPath`
- Added opened output folder - new automated time saver

see [LeanPubPandocConfigExecuteTest.java](#)

Walkthrough of Changes for Pandocifier Object

- I create an internal class called Pandocifier
- construct it in the @Before with the config
- amend the @Test to use a Pandocifier createPDF
- automatically create the createPDF
- copy paste the @Test code into createPDF

see [LeanPubPandocConfigExecuteObjectTest.java](#)

Walkthrough of Changes for External Config

- Use new @Ignore @Test to create a property file
- Create a property file reader that returns a config as an internal class
- Now the createPreviewMVP @Test method looks like a main method

see [LeanPubPandocConfigExecuteObjectPropertiesTest.java](#)

Make it an App

- create a main method, call from @Ignore MVP @Test
- move the inner classes out to main packages
- change the pom.xml to have a build section for the mainfest
- delete previous 'refactoring tests'
- mvn package
- because it is being released on github removed the 'leanpub' from .jar so people don't think it is official

Retrospective

- always kept the tactical mvp class running and available
- refactored to strategic solution
- inner classes until used by multiple classes e.g. main and MVP
- final strategic solution 'wrapped' as a 'tactical' @Ignored test
- didn't add many new features, didn't 'test', still a lot to do if 'other' people want to use it
- but now I can add it into CI for my local builds

Final Thoughts

- Automate tactically vs automate strategically
- Use of Tools
- Learn to program by extending tools
- How to choose a programming language
- How to move from tactical to strategic

Learn to code well

- still lots to learn, but its fun
- Abstractions
- Domain Driven Design
- Test Driven Development
- Design Patterns

Whatever you do. Make it add value.

Alan Richardson

- www.javafortesters.com
- www.seleniumsimplified.com
- www.eviltester.com
- www.compendiumdev.co.uk
- uk.linkedin.com/in/eviltester
- [@eviltester](#)